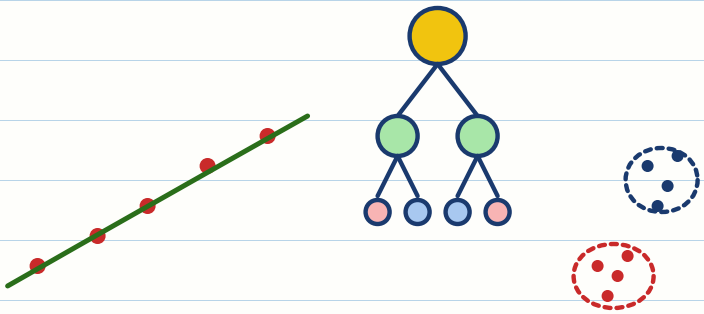
Machine Learning

~ handwritten notes ~



contents:

1. Regression · 2. Classification
3. Decision Trees · 4. SVM
5. PCA · 6. Clustering

– Sreemanti's notes ✏

1. Regression

Goal → predict a continuous output y from input features x.

✦ Linear Regression

The model assumes y is a linear combination of features:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = w^Tx + b$$

We learn weights w by minimising the Mean Squared Error (MSE):

$$MSE = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$$

Closed-form (normal eqn): $w^* = (X^TX)^{-1}\ X^Ty$

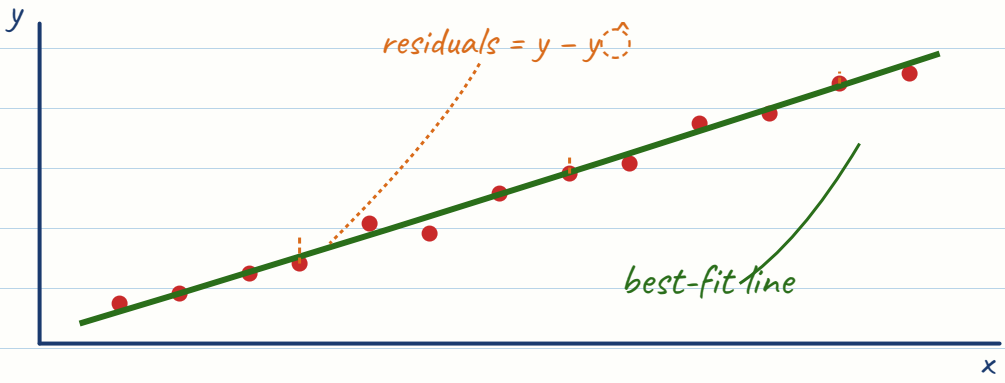
↑ works when X^TX is invertible & n is small.

✦ Gradient Descent

Iteratively update weights to descend the loss surface:

$$w \leftarrow w - \eta \cdot \nabla_w L(w)$$

η = learning rate. Too big → overshoots ✗ Too small → slow 🐢



✦ Regularization

To prevent overfitting, add a penalty on weights:

Ridge (L2):

$L = MSE + \lambda \Sigma w_i^2$

→ shrinks weights

Lasso (L1):

$L = MSE + \lambda \Sigma |w_i|$

→ sparse, feature selection!

✦ Evaluation Metrics

- MSE / RMSE — penalises big errors
- MAE — mean absolute error, robust to outliers
- $R^2 = 1 - SS_{res}/SS_{tot}$ ← variance explained

✦ Beyond Linear

► Polynomial regression

add x^2, x^3... as features → still linear in w!

► Logistic regression

squash output through sigmoid $\sigma(z) = 1/(1+e^{-z})$ for binary classification.

✓ Pros

- simple, interpretable
- fast to train
- baseline for any regression task

✗ Cons

- assumes linearity
- sensitive to outliers
- multicollinearity is a pain

★ *Worked example:* predict house price from sqft.

Data: (1000, $200k), (1500, $260k), (2000, $310k), (2500, $370k)

Fit → price ≈ 56 · sqft + 142,000

A 1800 sqft house ⇒ ≈ $243k □

2. Classification

Goal → predict a discrete label $y \in \{C_1, C_2, \ldots, C_k\}$.

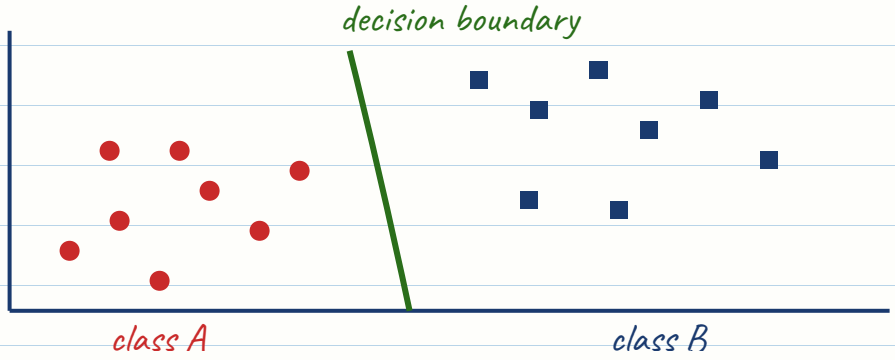
✦ Logistic Regression

Binary version: model $P(y=1 \mid x)$ via sigmoid.

$$P(y=1|x) = \sigma(w^T x + b), \quad \sigma(z) = 1 / (1 + e^{-z})$$

Loss = cross-entropy (a.k.a. log-loss):

$$L = -(1/N) \Sigma [\, y_i \log \hat{p}_i + (1-y_i) \log(1-\hat{p}_i) \,]$$



✦ Other Popular Classifiers

- k-NN — predict by majority vote of k nearest neighbours. Lazy learner.
- Naïve Bayes — Bayes rule + feature independence. Great for text □
- Decision Trees / RF — see §3
- SVM — see §4
- Neural Nets — deep, flexible, hungry for data

✦ Evaluation Metrics

► Confusion Matrix

	pred +	pred −
true +	TP	FN
true −	FP	TN

accuracy = (TP+TN)/all

precision = TP/(TP+FP)

recall = TP/(TP+FN)

F1 = 2·P·R/(P+R)

► ROC & AUC

Plot True Positive Rate vs False Positive Rate as threshold varies. AUC = 1 → perfect, 0.5 → random.

✦ Class Imbalance ⚠

If 99% of data is class A, predicting "always A" gives 99% accuracy but is useless!

> Fixes: resample (over/under), SMOTE, class weights, use F1 / PR-AUC instead of accuracy.

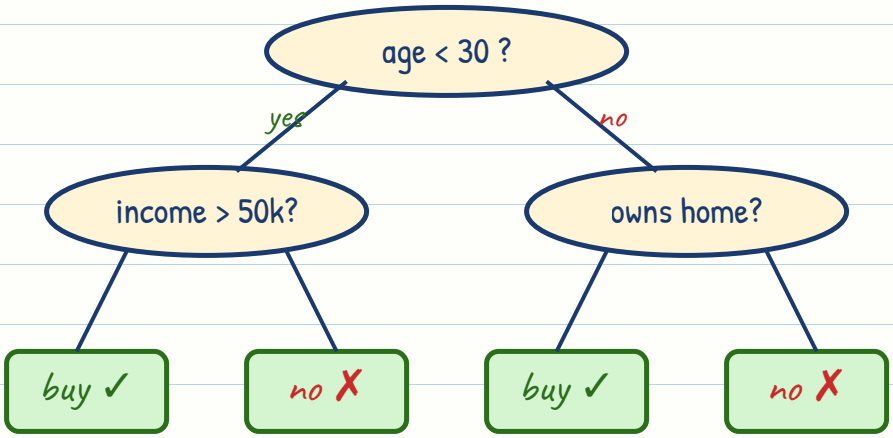
✦ Multiclass Strategies

- One-vs-Rest: k binary classifiers
- One-vs-One: $k(k-1)/2$ pairs
- Softmax: $P(y=c|x) = e^{z_c} / \Sigma e^{z_j}$

> ★ Example: spam filter. Features = word counts.
> Predict P(spam | email). Threshold at 0.5 → label.
> Tune threshold ↑ to reduce false alarms.

3. Decision Trees

Idea → recursively split feature space into rectangles based on simple yes/no questions until each region is "pure".



✦ How to Pick a Split?

Choose the split that maximises purity (or info gain).

Gini impurity:

$$G = 1 - \Sigma p_i^2$$

Entropy:

$$H = - \Sigma p_i \log_2 p_i$$

Information gain = $H(\text{parent}) - \Sigma\ (n_k/n) \cdot H(\text{child}_k)$

→ greedily pick split with highest IG (CART, ID3, C4.5).

✦ Stopping & Pruning

Trees love to overfit. Control with:

- max_depth, min_samples_split, min_samples_leaf
- *pre-pruning* — stop early
- *post-pruning* — grow big, trim back using validation set (cost-complexity α)

✦ Regression Trees

Same idea, but predict the mean of y in each leaf. Split criterion = variance reduction instead of Gini.

✦ Ensembles

► Random Forest

- build many trees on bootstrapped samples (bagging)
- at each split, consider random subset of features
- average predictions → low variance, robust

► Gradient Boosting (XGBoost, LightGBM)

- build trees sequentially; each fits residuals of the previous
- $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$
- often wins Kaggle 🏆

✓ Pros

- interpretable (single tree)
- handles mixed feature types
- no scaling needed
- captures non-linearity & interactions

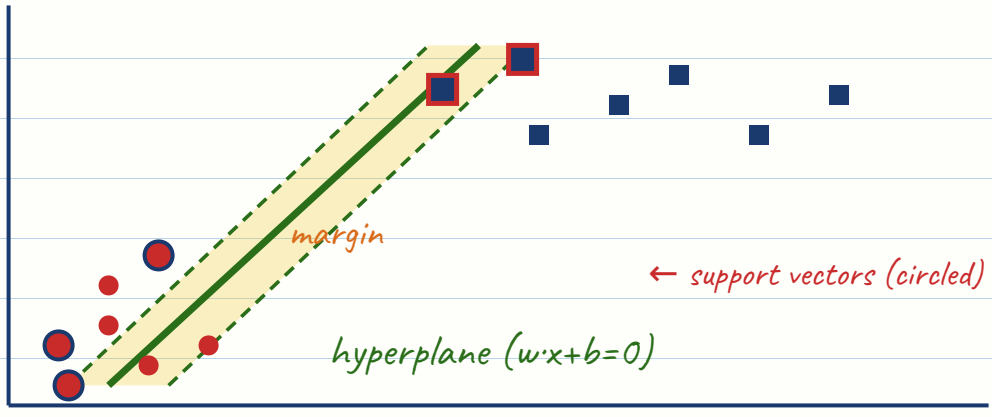
✗ Cons

- single tree → high variance
- unstable to small data changes
- greedy splits → not globally optimal
- poor for very smooth functions

★ *Intuition:* Decision trees draw axis-aligned rectangles. Random Forest = "wisdom of the crowd" of trees. Boosting = each tree apologises for the previous tree's mistakes ☹

4. Support Vector Machines

Idea → find the hyperplane that maximises the margin between two classes.



✦ Hard-Margin SVM

Assume data is linearly separable. Solve:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 \ \ \forall i$$

Margin width = $2 / \|w\|$ ← bigger is better!

✦ Soft-Margin SVM

Real data is messy — allow some violations via slack ξ_i:

$$\min \ \tfrac{1}{2}\|w\|^2 + C \sum \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i$$

C = penalty for mistakes. Big C → hard margin. Small C → soft & smooth.

✦ The Kernel Trick

What if data isn't linearly separable? Map $x \to \varphi(x)$ into a higher-dim space where it IS linear. But we never compute $\varphi(x)$ directly — just the kernel:

$$K(x, x') = \varphi(x)^T \varphi(x')$$

Linear:
$K = x^T x'$

Polynomial:
$K = (x^T x' + c)^d$

RBF (Gaussian):
$K = \exp(-\gamma \, ||x - x'||^2)$
↑ most popular!

Sigmoid:
$K = \tanh(\alpha \, x^T x' + c)$

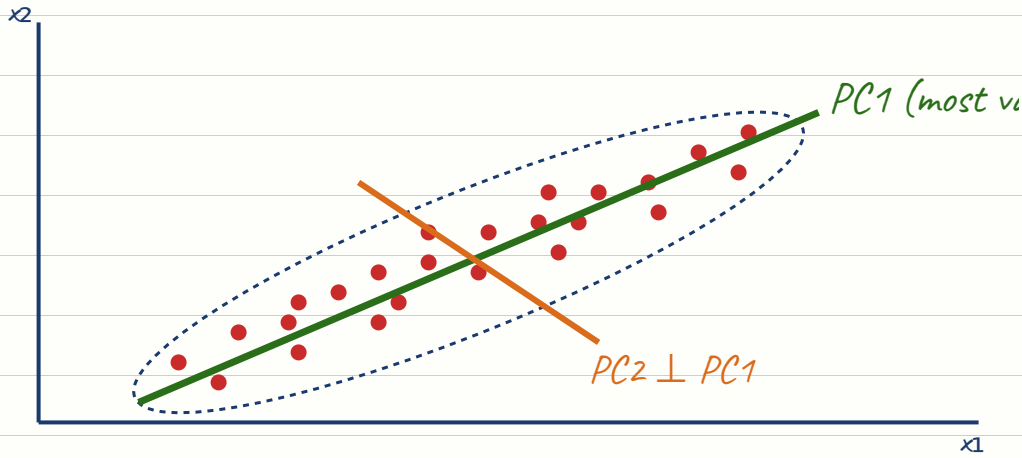


✓ Pros
- works well in high dimensions
- kernels → non-linear without explicit feature maps
- only support vectors matter → sparse

✗ Cons
- $O(N^2)$ to $O(N^3)$ — slow on big data
- kernel + C choice = lots of tuning
- no native probability output

5. PCA

Goal → reduce dimensionality by projecting data onto the directions of maximum variance.



✦ The Recipe

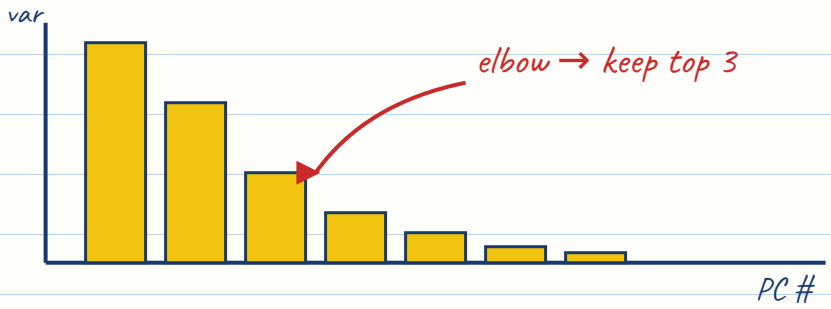
1. Centre the data: $x \leftarrow x - \bar{x}$ (subtract mean)
2. (optional) Scale: divide each feature by its std → use correlation matrix
3. Compute covariance matrix $\Sigma = (1/N)\, X^T X$
4. Find eigenvectors v_i & eigenvalues λ_i of Σ
5. Sort by λ (descending). Top k vectors = principal components
6. Project: $Z = X \cdot V_k$ → reduced data!

✦ Explained Variance

How much info does each PC carry?

$$\text{explained variance ratio} = \lambda_i \,/\, \Sigma\lambda_j$$

Plot the scree plot & pick k where the curve elbows.



✦ SVD Connection

For data matrix X (centred), the SVD is:

$$X = U\,\Sigma V^T \Rightarrow \text{ columns of } V = \text{principal components}$$

Singular values $\sigma_i \leftrightarrow \sqrt{(N \cdot \lambda_i)}$. SVD is numerically nicer than computing X^TX.

✦ When to Use PCA

- visualisation (project to 2D / 3D)
- noise filtering — drop tiny components
- speed up downstream ML
- de-correlate features

✓ Pros

- linear & deterministic
- reveals dominant structure
- fast (esp. with randomised SVD)

✗ Cons

- linear only — try t-SNE / UMAP for non-linear
- PCs aren't interpretable features
- sensitive to scale & outliers

★ *Remember:* always scale features before PCA if their units differ (e.g. height in cm vs salary in $). Otherwise the biggest-unit feature will dominate the first PC! □

6. Clustering

Goal → group similar points together without labels. Unsupervised learning!

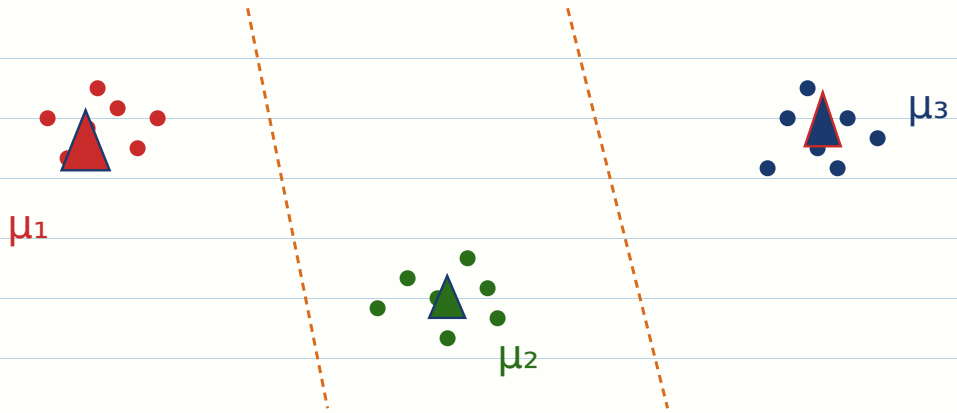
✦ K-Means

Partition N points into k clusters. Each cluster has a centroid μ_k.

minimise $\Sigma_k \Sigma_{x \in C_k} \|x - \mu_k\|^2$

Algorithm (Lloyd's):

1. pick k random centroids
2. assign each point to nearest centroid
3. update centroid = mean of its points
4. repeat 2 & 3 until centroids stop moving



▸ Choosing k

- Elbow method — plot inertia vs k, pick the elbow
- Silhouette — measures how well each point fits its cluster ($\in [-1, 1]$)
- Gap statistic — compare to random null

Tip: use k-means++ for smart initialisation.

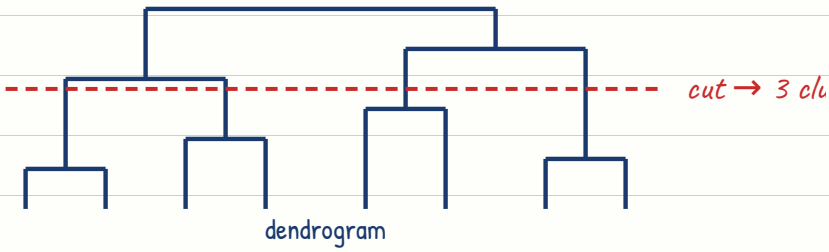
✦ Hierarchical Clustering

Build a tree (dendrogram) of clusters. Two flavours:

- Agglomerative (bottom-up): start with each point as a cluster, merge nearest pair until one remains
- Divisive (top-down): start with all in one, split recursively

Linkage options:

- single — min distance between clusters (chaining ✗)
- complete — max distance (compact clusters)
- average — mean pairwise distance
- Ward — minimise variance increase (popular ✓)



✦ DBSCAN

Density-based clustering — clusters = dense regions separated by sparse ones.

params: ε (radius), MinPts (density threshold)
core point: ≥ MinPts neighbours within ε
border: in neighbourhood of core but not itself dense
noise: everything else □

✓ finds arbitrary shapes, handles noise. ✗ tricky ε in varying density.

✦ Other Algorithms

- Gaussian Mixture Models (GMM) — soft clustering, EM algorithm, ellipsoidal clusters
- Spectral clustering — graph-based, uses eigenvectors of Laplacian
- Mean-shift — climb the density gradient

★ Rule of thumb: spherical & known k → K-means. Arbitrary shapes / noise → DBSCAN. Probabilistic → GMM. Want a hierarchy → agglomerative. □

Big Picture

Supervised — we have labels y

- continuous y → regression (lin/poly, ridge, lasso, regression trees, SVR, NN)
- discrete y → classification (logistic, k-NN, NB, tree, SVM, NN)

Unsupervised — no labels

- reduce dims → PCA, t-SNE, UMAP, autoencoders
- find groups → K-means, hierarchical, DBSCAN, GMM

Bias / Variance

total error = bias^2 + variance + irreducible noise

- high bias → underfit. (too simple model)
- high variance → overfit. (too complex)
- sweet spot via cross-validation + regularization.

ML Workflow

1. understand the problem & the data
2. clean & explore — handle missing, outliers
3. feature engineering (often the most impactful step!)
4. split: train / val / test
5. baseline model → iterate
6. tune hyperparams with CV
7. evaluate on test, deploy, monitor

★ Top tips:

- garbage in → garbage out. Spend time on data!
- always have a baseline (mean / majority class)
- inspect errors, not just metrics
- simpler models first → only go complex when needed
- the test set is sacred — touch only once

— end of notes —